УНИВЕРСИТЕТ ИТМО

С. А. Ким, О. А. Евстафьев

Прикладной искусственный интеллект

Лабораторный практикум



Санкт-Петербург 2026

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

УНИВЕРСИТЕТ ИТМО

С. А. Ким, О. А. Евстафьев

Прикладной искусственный интеллект

ЛАБОРАТОРНЫЙ ПРАКТИКУМ

Санкт-Петербург
2026

Ким С. А., Евстафьев О. А. Прикладной искусственный интеллект: лабораторный практикум. — Санкт-Петербург: Университет ИТМО, 2026. — 51 с.

Авторы: Ким Станислав Александрович; Евстафьев Олег Александрович.

Рецензент: Шаветов Сергей Васильевич, кандидат технических наук, доцент факультета систем управления и робототехники Университета ИТМО.

Практикум включает три работы: распознавание активности человека по признакам сигналов мобильных сенсоров; прогнозирование стоимости жилья полносвязной нейронной сетью; классификацию изображений цветов с использованием свёрточных сетей и переноса обучения. Для каждой работы приведены необходимые теоретические сведения, порядок выполнения, план исследования, контрольные точки и вопросы защиты.

Материал предназначен для студентов бакалавриата, изучающих дисциплину «Прикладной искусственный интеллект» и впервые знакомящихся с машинным обучением. Предполагаются навыки Python, работы с массивами и таблицами, знакомство со средним и производной. Основные понятия машинного обучения вводятся в самом практикуме. Сопровождающий комплект содержит учебные ноутбуки и код. Основные инструменты: scikit-learn, TensorFlow/Keras и PyTorch.

Особое внимание уделено корректному разделению данных, сопоставимости экспериментов, анализу ошибок и воспроизводимости результатов. Численные результаты опытов студент получает самостоятельно; в тексте приведены явно обозначенные расчётные примеры.

Об Университете ИТМО

Университет ИТМО — национальный исследовательский университет в Санкт-Петербурге. Его научные и образовательные направления включают информационные технологии и искусственный интеллект, фотонику, робототехнику, квантовые коммуникации, науки о жизни, искусство и научную коммуникацию. ИТМО участвует в программе «Приоритет-2030», развивает междисциплинарные исследования и индивидуальные образовательные траектории. С 2019 года университет самостоятельно присуждает учёные степени кандидата и доктора наук.

© Университет ИТМО, 2026

© Ким С. А., Евстафьев О. А., 2026

Содержание

Введение **4**

Первое знакомство с машинным обучением **6**

Подготовка среды и исходных материалов **8**

Практическая работа № 1. Классификация **10**
1.1 Постановка задачи и результат работы 10
1.2 От сигналов к признакам 10
1.3 Разделение данных по участникам 11
1.4 Три семейства классификаторов 12
1.5 Метрики и матрица ошибок 13
1.6 Порядок выполнения 13
1.7 Отчёт и вопросы защиты 16

Практическая работа № 2. Нейронные сети **18**
2.1 Постановка задачи и результат работы 18
2.2 Обучение, валидация и данные для прогнозирования 18
2.3 Пропуски, категории и масштабирование 19
2.4 Полносвязная нейронная сеть 19
2.5 Функции активации 20
2.6 Функции потерь и метрики регрессии 21
2.7 Обучение: градиент, эпоха и мини-выборка 22
2.8 Переобучение и воспроизводимость 23
2.9 Порядок выполнения 23
2.10 Выполнение на PyTorch 27
2.11 Самостоятельное исследование гиперпараметров 28
2.12 Типичные ошибки и способы проверки 30
2.13 Отчёт и защита работы № 2 31

Практическая работа № 3. Свёрточные нейронные сети **33**
3.1 Постановка задачи и результат работы 33
3.2 Изображение как тензор 33
3.3 Свёрточный слой 34
3.4 Pooling и устройство исходной сети 34
3.5 Логиты, cross-entropy и обучение 35
3.6 Перенос обучения и заморозка 36
3.7 Подготовка изображений и разбиение 37
3.8 Обучение и исследование архитектур 38
3.9 Сравнение, визуализация и сохранение 39
3.10 Контрольные вопросы и критерии готовности 41

Общие требования к отчёту и рекомендации преподавателю **43**

Современные модели для таблиц: дополнительное чтение **45**

Литература **47**

Приложение А. Материалы эксперимента **49**

Приложение Б. Шаблон отчёта **51**

Введение

Практикум подготовлен по дисциплине «Прикладной искусственный интеллект». Искусственный интеллект объединяет методы решения задач, для которых нужны распознавание, прогнозирование или принятие решений. Машинное обучение — одно из его направлений: модель находит закономерности по примерам. Три работы этого практикума посвящены именно машинному обучению и нейронным сетям на Python.

Практикум учит проходить полный цикл работы: понять данные, определить проверяемую задачу, построить модель, провести сопоставимые эксперименты и объяснить ошибки. Рабочий код служит инструментом исследования. Итог работы — вывод, который можно проверить по сохранённым данным и результатам.

Три работы связаны общей логикой, но используют разные исходные данные и разные виды ответа. В первой модели получают уже вычисленные признаки сигналов и возвращают один из шести классов активности. Во второй сеть предсказывает непрерывную цену. В третьей признаки изображения формируются внутри свёрточной сети, а ответом служит вид цветка.

Таблица 1: Карта практикума

№	Задача	Главное новое умение	Проверяемый результат
1	Классификация активности	Сравнение алгоритмов; разделение по участникам	Отчёт по шести классам и матрица ошибок
2	Регрессия стоимости	Подготовка смешанных признаков; обучение полносвязной сети	MAE/RMSE, анализ ошибок, таблица прогнозов
3	Классификация цветов	Свёртки, перенос обучения, управление режимами модели	Сравнение трёх архитектур на одной валидации

Входная подготовка

Предшествующий курс машинного обучения не требуется. Нужны функции и коллекции Python, чтение таблиц в pandas, простые операции с массивами NumPy, среднее и представление о производной. Вектор можно сначала понимать как список чисел, матрицу — как прямоугольную таблицу. Смысл новых понятий объясняется перед их использованием; формулы помогают проследить вычисление.

Начните с раздела «Первое знакомство с машинным обучением». Работы выполняйте по порядку: первая вводит сравнение моделей, вторая — обучение нейросети, третья — работу с изображениями. Обзор TabM и TabPFN-3 в конце практикума предназначен для дополнительного чтения.

Общий порядок работы

1. Изучите исходные файлы и смысл меток. Проверьте размеры, пропуски, классы и идентификаторы.
2. Зафиксируйте разбиение, метрики, конфигурации и бюджет до сравнения результатов.
3. Выполните простой исходный вариант и проверьте контрольные точки.
4. Меняйте заранее выбранный фактор, сохраняйте результаты каждого опыта.
5. Объясните характерные ошибки и сформулируйте границы вывода.

> **Как читать примеры кода**
>
> Сначала назовите вход и ожидаемый результат ячейки, затем выполните её и объясните полученное. Учебный модуль — обычный файл Python с функциями: его можно открыть и прочитать. На первом проходе важнее понять роль функции; затем изучите нужные строки её реализации. Самостоятельная часть — выбор, изменение модели и объяснение наблюдений.

Роль обучающей, валидационной и тестовой частей

Обучающая часть определяет параметры модели и статистики преобразований. Валидация помогает выбрать настройки. Независимый размеченный тест используют после завершения выбора. Название файла само по себе не определяет его роль: в работе № 1 `test.csv` содержит правильные ответы, а в работе № 2 — только признаки новых домов.

Подбор по одной и той же валидации постепенно приспосабливает к ней эксперимент. Поэтому в работах № 2 и № 3 результаты на валидации описывают качество выбора в данном опыте; они не объявляются независимой итоговой оценкой. В работе № 1 предусмотрен отдельный финальный тест.

Первое знакомство с машинным обучением

От примеров к правилу

Предположим, в таблице записаны площадь, число комнат и известная цена нескольких домов. Мы хотим оценить цену другого дома. В машинном обучении правило прогноза настраивают по примерам с известным ответом. Эти примеры называются **обучающими данными**, а настраиваемое правило — **моделью**.

Понятие	Пример и смысл
Объект	Один дом; в таблице ему соответствует одна строка.
Признак	Известная характеристика дома, например площадь.
Целевой ответ	То, что требуется предсказать: здесь цена.
Выборка	Набор объектов, выделенный для определённой цели.
Прогноз	Ответ модели для конкретного объекта.
Обучение	Настройка модели по признакам и известным ответам.

При **регрессии** ответ — число, например цена. При **классификации** ответ — одна из заранее заданных категорий, например «ходит» или «сидит». Категории называют классами, а правильный класс объекта — его меткой. Числовой код класса служит обозначением и не превращает задачу в регрессию.

Во всех трёх работах рассматривается **обучение с учителем**: для обучающих примеров известны правильные ответы. «Учитель» здесь означает наличие ответов в данных. Для нового объекта модель получает только те сведения, которые доступны при её применении.

Как читать обозначения

Запись x_i обозначает признаки объекта с номером i, y_i — правильный ответ, $\widehat{y}_i$ — прогноз. Например, $x_i = (80, 3)$ может описывать площадь и число комнат. Символ f_θ обозначает модель, а θ — её настраиваемые параметры:

$$\widehat{y}_i = f_\theta(x_i).$$

Эта формула читается так: «подать признаки объекта в модель и получить прогноз». В работе с изображениями признаки на входе — значения пикселей; в работе с активностью — уже вычисленные характеристики сигналов.

Почему нельзя проверять только на знакомых примерах

Правило может хорошо отвечать на знакомые примеры и ошибаться на новых. Способность работать на новых объектах называют **обобщением**. Если модель слишком подстроилась под особенности обучения и хуже переносится на новые данные, говорят о **переобучении**.

Для организации проверки данные разделяют по ролям:

1. **Обучение (train):** настраиваем модель.
2. **Валидация (validation):** сравниваем варианты и выбираем настройки.
3. **Итоговый тест (test):** оцениваем выбранное решение, если есть отдельные данные с ответами, не использованные при выборе.

Можно представить подготовку к экзамену: разобранные задачи помогают учиться, пробный вариант — выбирать способ подготовки, новый экзаменационный вариант — проверять результат. Это аналогия ролей, а не требование делить любой набор ровно на три файла.

Утечка данных возникает, если при обучении используется информация, которая по условиям проверки должна быть недоступна. Например, цена дома случайно оставлена среди признаков или средние для подготовки признаков вычислены с использованием валидации.

Что именно меняет экспериментатор

Параметры модель находит при обучении: например, веса нейросети. **Гиперпараметры** задаёт исследователь: например, число соседей или ширину скрытого слоя. **Метрика** — число, которым оценивают качество прогноза. **Исходная модель (baseline)** — понятный вариант, с которым сравнивают изменения.

Под **протоколом** понимается заранее записанный порядок опыта: какие данные и разбиение используются, что меняется, какая метрика сравнивается. **Бюджет** ограничивает число запусков, эпох или время. **Seed** — начальное состояние генератора случайных чисел: оно помогает повторить выбор разбиения и начальных весов, но не делает оценку качества безошибочной.

> **Проверьте себя перед первой работой**
>
> Выберите один дом или одно изображение. Назовите объект, доступные признаки и искомый ответ. Объясните, чем настройка модели отличается от проверки её прогноза. Если ответ пока неясен, вернитесь к этим определениям до запуска кода.

Подготовка среды и исходных материалов

Электронное приложение к практикуму:
`https://github.com/stalkim/itmo-ml-practicum`

Структура файлов

Держите исходные данные разных работ в отдельных каталогах. Файлы с одинаковыми именами относятся к разным задачам и не взаимозаменяемы.

```
project/
  code/                          # common teaching modules
  labs/
    01_classification/
      lab1_activity.ipynb
      data/                      # train.csv, test.csv
    02_regression/
      lab2_housing.ipynb
      data/                      # CSV and data_description.txt
    03_cnn/
      lab3_flowers.ipynb
      data/                      # five class folders
```

Работа	Что должно быть в данных
№ 1: активность	В обеих таблицах: числовые признаки, метка `Activity`, идентификатор участника (в коде — `subject`).
№ 2: жильё	В обучающей таблице: `Id`, признаки и `SalePrice`; в таблице прогнозирования цены нет. Описание признаков находится в `data_description.txt`.
№ 3: цветы	Пять каталогов видов с изображениями. Индексы классов определяются фактическим отображением `class_to_idx`.

Преподаватель размещает выборки в папках работ или выдаёт их отдельно. Если в `data` пока только инструкция, добавьте необходимые файлы. Набор изображений указан в исходном задании: Flowers Recognition. Количество строк, признаков и изображений определяйте по своей версии файлов.

Что такое ноутбук и как его запускать

Ноутбук Jupyter (файл `.ipynb`) состоит из текстовых ячеек с пояснениями и ячеек Python. Выполняйте их сверху вниз. Переменные предыдущих ячеек остаются в памяти: если выполнить только нижнюю ячейку после изменения данных, можно случайно использовать старый результат. Перед итоговой проверкой перезапустите среду исполнения (kernel/runtime) и выполните все ячейки заново.

GitHub показывает содержимое ноутбука и хранит файлы; код исполняется в Jupyter Notebook или JupyterLab на компьютере студента либо учебном сервере. Для этого комплекта

нужен весь проект с папкой `code`, а не только один `.ipynb`. В `START_HERE.md` электронного приложения приведён порядок первого запуска. На GitHub сохраняйте себе копию материалов согласованной версии.

Программная среда

Для работ № 1 и № 2 нужны Python, NumPy, pandas, scikit-learn и Matplotlib; для второго примера также TensorFlow/Keras. Для работы № 3 нужны PyTorch, совместимая с ним torchvision и Pillow. Инструкции по проверенному локальному окружению находятся в `code/README.md`.

CPU — центральный процессор компьютера. GPU — ускоритель, который может быстрее выполнять многие операции нейросетей; его наличие не меняет смысла задания. В готовом учебном окружении сначала проверьте версии библиотек. При самостоятельной установке согласуйте версии Python, PyTorch, torchvision и доступного ускорителя по документации. Для работы на GPU нужна подходящая сборка библиотек.

```
import sys
from importlib.metadata import version
print(sys.version)
for name in ("numpy", "pandas", "scikit-learn"):
    print(name, version(name))
```

В третьей работе ускоритель существенно сокращает время обучения. Проверка `torch.cuda.is_available()` сообщает о доступности CUDA, а не о достаточном объёме памяти. Короткую проверку небольшого варианта можно выполнить на CPU; для сравнения тяжёлой исходной сети заранее оцените ресурсы.

Пути в ноутбуке

Первая ячейка кода находит корень проекта по папкам `code` и `labs`, затем выбирает каталог своей работы. Поэтому путь `data/train.csv` ведёт к данным именно этой работы, а `runs` — к её результатам. Запускать ноутбук можно из корня проекта или каталога работы. Сохраняйте структуру скачанного комплекта; для нового опыта выбирайте новое имя папки результатов.

> **Перед запуском**
>
> Проверьте, что открываете данные нужной работы; можете прочитать таблицу или изображение; понимаете смысл целевого ответа; знаете, какие файлы будут сохранены. Зафиксируйте версии библиотек и seed. Не сравнивайте результаты, полученные на незаметно разных разбиениях.

Практическая работа № 1. Классификация

1.1. Постановка задачи и результат работы

По признакам сигналов смартфона требуется определить вид активности человека. Объектом является временное окно измерений — короткий фрагмент записи. Оно представлено вектором $x_i \in \mathbb{R}^d$: списком из d числовых характеристик. Модель возвращает один класс:

$$\widehat{y}_i = f_\theta(x_i), \qquad y_i \in \{0, 1, 2, 3, 4, 5\}. \tag{1.1}$$

Числа обозначают категории: разность между метками не измеряет сходство движений. Устройство модели и её гиперпараметры студент выбирает и объясняет; в качестве сравнимого исходного набора ниже предложены три семейства алгоритмов.

Таблица 2: Словарь меток активности

Код	Метка в файле	Смысл
0	`WALKING`	Ходьба
1	`WALKING_UPSTAIRS`	Ходьба вверх по лестнице
2	`WALKING_DOWNSTAIRS`	Ходьба вниз по лестнице
3	`SITTING`	Сидение
4	`STANDING`	Стояние
5	`LAYING`	Положение лёжа

Результат сдачи: таблица сравнения моделей на валидации, обоснованный выбор, итоговый `classification_report` на тесте, матрица ошибок, визуализация сравнения и объяснение влияния гиперпараметров. По завершении работы вы должны уметь различать precision и recall, читать показатели по отдельным классам и объяснять, почему случайное разделение строк бывает недостаточным.

1.2. От сигналов к признакам

Описание исходного задания соответствует набору UCI Human Activity Recognition Using Smartphones. В первоисточнике 30 участников выполняли шесть действий со смартфоном на поясе; акселерометр и гироскоп регистрировали сигналы с частотой 50 Гц. Окна длиной 2,56 с содержали 128 отсчётов и перекрывались на 50%. Из окна извлекались 561 признак во временной и частотной областях [1]. Число столбцов в учебной CSV-выгрузке проверяется отдельно.

Акселерометр измеряет ускорение, гироскоп — угловую скорость. Фильтрация ослабляет шум; отделение низкочастотной составляющей помогает различать вклад гравитации и движения тела. Производные сигналов описывают скорость изменения, а евклидова норма объединяет три оси в скалярную величину. Преобразование Фурье показывает, как энергия сигнала распределяется по частотам. В этой работе не требуется заново реализовывать обработку сырых сигналов: используются готовые признаки.

Таблица 3: Как читать обозначения признаков исходного задания

Обозначение	Что оно помогает описать
`tBodyAcc`, `tGravityAcc`, `tBodyGyro`	Ускорение тела, гравитационная составляющая, угловая скорость; `t` обозначает временную область.
`Jerk`, `Mag`, `XYZ`	Изменение сигнала, его модуль, отдельные оси.
Префикс `f`	Представление в частотной области после БПФ.
`mean`, `std`, `mad`, `iqr`	Характеристики положения и разброса; точное определение проверяется по описанию признаков.
`max`, `min`, `sma`, `energy`, `entropy`	Экстремумы, совокупная амплитуда, энергия и характеристика распределения сигнала.
`arCoeff`, `correlation`, `angle`	Динамическая структура, связь сигналов, геометрическое соотношение векторов.
`maxInds`, `meanFreq`, `bandsEnergy`, `skewness`, `kurtosis`	Положение частотных компонентов, энергия полос и форма распределения.

Не приписывайте признаку смысл только по имени. Например, величина `energy` зависит от принятого нормирования. Для последовательности $a_1, \ldots, a_N$ удобный пример нормированной энергии — $N^{-1} \sum_i a_i^2$; это не то же самое, что сумма без деления на N. В отчёте разберите два конкретных столбца своей таблицы.

1.3. Разделение данных по участникам

Если соседние окна одного человека попадают и в обучение, и в проверку, модель может воспользоваться индивидуальными особенностями и близостью записей. Результат тогда не обязательно переносится на нового участника. Для данного учебного протокола единица разделения — **человек**, а единица классификации — **окно сигнала**.

Сохраняйте внешний `test.csv` для итоговой проверки. Внутри `train.csv` выделите валидацию по идентификатору участника: примерно 20% участников, фиксированный seed 40. `GroupShuffleSplit` определяет долю групп, поэтому доля строк может отличаться от 20% [2]. Проверьте наличие всех шести классов в каждой части. Не выбирайте seed по наилучшей метрике.

> **Три непересекающихся множества**
>
> Участники обучения и валидации не пересекаются; участники внешнего теста отсутствуют в обеих этих частях. Столбцы `Activity` и `subject` не поступают в классификатор. Если идентификатора участника нет, нельзя утверждать, что проверена работа на новых людях: сначала уточните состав выгрузки и протокол.

Числовые признаки приводят к сопоставимому масштабу, например $\widetilde{x}_j = (x_j - \mu_j)/s_j$. Среднее и масштаб вычисляются только на обучающей части. `Pipeline` связывает преобразование с моделью и предотвращает случайное применение нового scaler к проверочным объектам [3, 4].

1.4. Три семейства классификаторов

Каждый алгоритм строит своё правило ответа. Сначала разберите его идею словами, затем найдите в коде настройку, которая меняет это правило. Реализовывать алгоритмы библиотек самостоятельно не требуется.

Метод ближайших соседей

Для нового вектора находятся k ближайших обучающих объектов; класс определяется голосованием. При евклидовом расстоянии

$$\rho(x, z) = \sqrt{\sum_{j=1}^{d} (x_j - z_j)^2} \tag{1.2}$$

крупный масштаб одного признака может подавить вклад остальных. Поэтому в исходном сравнении KNN используется после стандартизации. Малое k делает ответ чувствительным к отдельным объектам; большое сильнее сглаживает границу. Существенны также способ взвешивания соседей и метрика расстояния [5].

Линейный метод опорных векторов

При двух признаках линейную границу можно представить прямой на плоскости: точки по разные стороны получают разные ответы. Для большего числа признаков действует та же идея разделения. Метод опорных векторов (SVM) стремится получить границу с широким зазором между классами, допуская штрафуемые нарушения. Параметр C управляет компромиссом между регуляризацией и штрафом за нарушения разделения: уменьшение C усиливает регуляризацию — ограничение сложности подгонки к обучению. Это не универсальное правило улучшения качества. В исходном сравнении используется `LinearSVC`; реализация линейного метода опорных векторов [6].

Случайный лес

Дерево последовательно задаёт вопросы о признаках, например «значение больше порога?», и в конце выдаёт класс. Случайный лес (Random Forest) объединяет ответы многих деревьев, обученных со случайным выбором объектов и доступных признаков. Ограничение глубины меняет сложность отдельных деревьев; число деревьев влияет на устойчивость ансамбля и вычислительные затраты. Масштабирование признаков для пороговых разбиений дерева обычно не требуется; для леса в предложенном протоколе scaler не добавляется [7].

Гиперпараметры задаются до обучения конкретной модели: k, C, глубина и число деревьев. Параметры, например коэффициенты линейной модели, определяются при `fit`. Изменение гиперпараметров требует нового обучения. Для контроля добавляется

`DummyClassifier`, всегда предсказывающий самый частый класс обучения. Он показывает результат без анализа признаков: сравнение с ним помогает понять, чему научились остальные модели.

1.5. Метрики и матрица ошибок

Пусть C_{ij} — число объектов истинного класса i, отнесённых к классу j. Строки матрицы ошибок соответствуют истине, столбцы — прогнозу. Для класса c:

$$TP_c = C_{cc}, \qquad FP_c = \sum_{i \neq c} C_{ic}, \qquad FN_c = \sum_{j \neq c} C_{cj}, \tag{1.3}$$

$$P_c = \frac{TP_c}{TP_c + FP_c}, \qquad R_c = \frac{TP_c}{TP_c + FN_c}, \qquad F_{1,c} = \frac{2TP_c}{2TP_c + FP_c + FN_c}. \tag{1.4}$$

Precision отвечает на вопрос, какая доля предсказаний класса c верна. Recall показывает, какая доля настоящих объектов класса c найдена. F1 объединяет эти характеристики; высокая precision сама по себе не означает высокую полноту.

$$\text{Accuracy} = \frac{\sum_c C_{cc}}{n}, \qquad \text{MacroF1} = \frac{1}{6}\sum_{c=0}^{5} F_{1,c}. \tag{1.5}$$

Macro-F1 придаёт каждому классу одинаковый вес. Weighted-F1 взвешивает классы по их численности; support — число истинных объектов класса. Для выбора в этой работе используйте macro-F1 валидации, а accuracy и показатели по классам — для дополнительного анализа [8].

> **Пример для самопроверки**
>
> Для класса «сидит» модель дала 10 положительных прогнозов: восемь верны, два относятся к другим классам; ещё четыре объекта класса «сидит» пропущены. Тогда precision равна $8/10 = 0{,}8$, recall — $8/12 \approx 0{,}667$, F1 — $16/22 \approx 0{,}727$. Это вычислительный пример, не результат работы на данных курса.

При нулевом знаменателе метрика не определена обычной формулой. В коде `zero_division=0` задаёт явное соглашение вывода; нулевой support или отсутствие предсказаний следует отметить. Нормирование матрицы по строкам помогает увидеть долю ошибок для каждого истинного класса, а абсолютные числа показывают объём наблюдений.

1.6. Порядок выполнения

Первый проход: прочитайте таблицу, назовите её служебные столбцы, затем выполните исходное сравнение без изменения настроек. Найдите одну строку результата и объясните её метрику. После этого переходите к исследованию гиперпараметров.

Шаг 1. Проверьте таблицы и метки

Откройте файлы из `data`. Выведите размер таблиц, имена столбцов, число участников, распределение классов и наличие пропусков. Назовите два осмысленных признака. Не выбирайте входы срезом `values[:, :-2]`: положение служебных столбцов может измениться.

```
from pathlib import Path
import pandas as pd
train = pd.read_csv(Path("data/train.csv"))
test = pd.read_csv(Path("data/test.csv"))
print(train.shape, test.shape)
print(train["Activity"].value_counts())
print(train["subject"].nunique(), test["subject"].nunique())
```

Шаг 2. Выделите валидацию

Учебный модуль проверяет метки и признаки, исключает целевой столбец и идентификатор участника, затем выполняет групповое разделение. Если идентификатор называется иначе, укажите его через `group_column`; не создавайте фиктивные номера участников из номеров строк.

```
from lab1_utils import load_har, candidate_models, fit_validation
from lab1_utils import CLASSES, final_test, save_har_result
data = load_har("data", group_column="subject", seed=40)
print(data["Xtr"].shape, data["Xva"].shape)
print(data["training_subjects"], data["validation_subjects"])
```

> **Контрольная точка**
>
> Во входах нет `Activity` и идентификатора участника. Обучение, валидация и внешний тест содержат все шесть классов и разные группы людей. Валидация не участвует в вычислении параметров scaler. Столбцы теста приведены к порядку обучающих признаков по именам.

Шаг 3. Объясните выбор и выполните сравнение

Сначала сформулируйте ожидаемое влияние гиперпараметров. В предлагаемом минимуме девять конфигураций с учётом константной модели. Допускается иной набор нескольких алгоритмов с сопоставимым и заранее определённым объёмом исследования.

Таблица 4: Исходный план сравнения классификаторов

Обозначения	Модель	Меняемый параметр
D0	Dummy	Самый частый класс; ориентир без использования признаков
K3, K7, K15	KNN	$k \in \{3, 7, 15\}$; одинаковое масштабирование
S0.1, S1, S10	LinearSVC	$C \in \{0{,}1, 1, 10\}$; остальные настройки фиксированы
R8, RNone	Random forest	Глубина 8 или без ограничения; 200 деревьев

```
models = candidate_models(seed=40)
rows = []
for name, estimator in models.items():
    result = fit_validation(data, estimator)
    rows.append({"model": name, **result["metrics"],
                 "seconds_fit": result["seconds_fit"]})
comparison = pd.DataFrame(rows)
print(comparison.sort_values("macro_f1", ascending=False))
```

Этот цикл использует только обучение и валидацию. Сохраняйте предупреждения о сходимости; не скрывайте их общим отключением warnings. Если модель не сошлась, выясните причину и отдельно зафиксируйте изменённый бюджет или настройку.

Шаг 4. Визуализируйте и зафиксируйте выбор

Постройте столбчатую диаграмму macro-F1 для всех конфигураций и отдельную диаграмму времени обучения. Не изображайте секунды и F1 как величины одной шкалы. Назовите худшие классы и предполагаемые причины ошибок. При близких результатах учитывайте сложность и время; не объявляйте небольшое различие устойчивым без повторной проверки.

```
import matplotlib.pyplot as plt
comparison.plot.bar(x="model", y="macro_f1", legend=False)
plt.ylabel("Validation macro-F1")
plt.tight_layout()
```

Шаг 5. Выполните итоговый тест

После выбора конфигурации переобучите её на всём `train.csv`; scaler внутри Pipeline также обучается заново. Только после этого получите прогноз `yhat` для `test.csv`. Пример ниже выбирает максимум macro-F1 валидации; окончательное объяснение выбора должно присутствовать в отчёте.

```
selected = comparison.sort_values(
    ["macro_f1", "model"], ascending=[False, True]
).iloc[0]["model"]
final = final_test(data, models[selected])
yhat = final["prediction"]
print(pd.DataFrame(final["report"]).T)
save_har_result(data, comparison, selected, final, "runs/lab1_01")
```

Если результат теста оказался хуже ожиданий, опишите это как результат. Возвращение к выбору настроек после просмотра теста превращает его в ещё одну валидацию. Исходное требование сравнить несколько моделей выполняется на валидации; внешний тест оценивает уже выбранное решение.

Шаг 6. Разберите ошибки

Постройте матрицу ошибок с одинаковым порядком шести меток. Выведите абсолютные числа и вариант с нормированием по строкам. Объясните хотя бы две заметные пары перепутанных действий; разделяйте наблюдение и гипотезу о его причине.

```
from sklearn.metrics import ConfusionMatrixDisplay
ConfusionMatrixDisplay.from_predictions(
    data["ytest"], yhat, labels=range(6),
    display_labels=CLASSES, normalize="true", xticks_rotation=90
)
plt.tight_layout()
```

1.7. Отчёт и вопросы защиты

В отчёт включите паспорт файлов, состав разбиения по участникам, мотивацию выбора алгоритмов, таблицу всех конфигураций, две диаграммы, итоговый classification report и матрицу ошибок. Сохраните точные параметры модели, версии, seed и идентификаторы групп. Завершите выводом: какой вариант выбран, какие классы распознаются хуже и что можно проверить следующим опытом.

1. Что является объектом, признаком, целевой меткой и группой в этой работе?
2. Почему код активности нельзя трактовать как непрерывную величину?
3. Какую информацию дают акселерометр и гироскоп? Зачем нужны признаки частотной области?
4. Почему разбиение строк и разбиение участников проверяют разные условия применения?
5. Чем precision отличается от recall? Постройте собственный пример с FP и FN.
6. Как рассчитывается F1? Чем macro-F1 отличается от weighted-F1 и accuracy?
7. Чем гиперпараметр отличается от параметра обученной модели?
8. Как изменение k, C и глубины деревьев повлияло на ваши результаты?
9. Почему scaler обучается только на обучающей части? Для каких моделей он особенно существенен?

10. Что означает нулевой support? Почему следует сохранять порядок меток в отчёте?
11. Почему нельзя выбрать победителя по внешнему тесту и затем назвать этот тест независимым?

> **Перед сдачей работы № 1**
>
> Есть сравнение нескольких моделей с вариациями гиперпараметров, визуализация, финальные предсказания и объяснение precision/recall/F1. Проверка отделена от подбора; `subject` не используется как входной признак. Указаны ограничения проверки на новых людях.

Практическая работа № 2. Нейронные сети

2.1. Постановка задачи и результат работы

По характеристикам дома необходимо предсказать его стоимость `SalePrice`. Пусть x_i — вектор подготовленных признаков, y_i — известная цена, а f_θ — модель с обучаемыми параметрами θ. Тогда прогноз имеет вид

$$\widehat{y}_i = f_\theta(x_i), \qquad y_i \in \mathbb{R}_{>0}. \tag{2.1}$$

Требуется подготовить данные, построить последовательную нейронную сеть, исследовать её настройки, оценить результат на выделенной валидационной части и получить прогнозы для домов из `test.csv`.

Результат сдачи: выполненный ноутбук, таблица экспериментов, диагностические графики, обоснованный выбор конфигурации и таблица `Id, SalePrice`. Отдельно сохраняются исходные данные эксперимента и сведения, позволяющие его повторить.

2.2. Обучение, валидация и данные для прогнозирования

В исходном файле `train.csv` известны правильные ответы. Разделим его строки на обучающую часть (80%) и валидационную (20%). Первую используем для определения параметров преобразований и весов сети, вторую — для сравнения конфигураций. В `test.csv` ответов нет.

> **Основное правило**
>
> Медианы, средние, масштабы и словарь категорий определяются по обучающей части. К валидации и новым домам применяются уже найденные преобразования. В терминах scikit-learn: `fit_transform` для обучения, `transform` для остальных данных [3].

Если сначала вычислить медиану по всему `train.csv`, а затем разделить строки, в подготовку обучения попадёт информация о валидации. Если отдельно закодировать категории в двух таблицах, один и тот же числовой код может означать разные категории. Оба случая нарушают условия корректного сравнения.

Случайное разделение подходит для учебного предположения о независимых однородных наблюдениях. Для временного прогноза, повторных продаж одного объекта или групп зависимых домов нужен другой способ разбиения. Укажите применяемое предположение в отчёте.

Повторный выбор модели по одной валидации адаптирует решение к ней. Полученные числа являются оценкой на использованном разбиении, а не независимой финальной проверкой. Для такой проверки понадобилась бы отдельная размеченная выборка, не участвовавшая в выборе настроек.

2.3. Пропуски, категории и масштабирование

Для числового признака в исходном варианте будем заменять пропуски медианой обучающей части. Медиана меньше реагирует на отдельные очень большие значения, чем среднее. Выбранное правило одинаково применяется ко всем частям данных [9].

Для категорий введём отдельное значение `__missing__`. По описанию данных необходимо проверить, означает ли отсутствие записи неизвестное значение или отсутствие самого объекта: например, гаража. Автоматически отождествлять эти случаи нельзя. В исходном варианте сохраняется фиксированное удаление пяти столбцов из задания: `Alley`, `FireplaceQu`, `PoolQC`, `Fence`, `MiscFeature`. Это условие учебного сравнения, а не универсальное правило обработки пропусков.

Категории без естественного порядка кодируем методом *one-hot*: каждой известной категории соответствует отдельный бинарный столбец. Порядок столбцов задаётся кодировщиком, обученным один раз. `handle_unknown="ignore"` позволяет обработать ранее не встречавшуюся категорию: её блок индикаторов будет нулевым [10]. Это не даёт модели знаний о новой категории; её появление стоит отметить.

> **Небольшой пример**
>
> Для категорий A, B, C возможны коды $A \mapsto (1,0,0)$, $B \mapsto (0,1,0)$, $C \mapsto (0,0,1)$. Замена их числами $1, 2, 3$ навязывает порядок и расстояния, которых в описании признака может не быть.

Числовые признаки приведём к сопоставимому масштабу:

$$\widetilde{x}_{ij} = \frac{x_{ij} - \mu_j}{s_j}, \tag{2.2}$$

где μ_j, s_j вычислены только по обучающей части после заполнения пропусков. Для постоянного признака реализация использует безопасный масштаб 1. Масштабирование не делает признаки независимыми и не превращает распределение в нормальное [4].

В примере также стандартизуется целевая цена:

$$z_i = \frac{y_i - \mu_y}{s_y}, \qquad \widehat{y}_i = \mu_y + s_y \widehat{z}_i. \tag{2.3}$$

Это помогает обучать сеть на числах умеренного масштаба. Перед оценкой и сохранением прогноз возвращается к исходной цене. Использовать среднюю цену валидационной части при обратном преобразовании нельзя.

2.4. Полносвязная нейронная сеть

Нейрон полносвязного слоя вычисляет взвешенную сумму и применяет функцию активации:

$$a_j = \sum_{k=1}^{d} w_{jk} x_k + b_j, \qquad h_j = \varphi(a_j). \tag{2.4}$$

В матричной записи для слоя ℓ:

$$h^{(\ell)} = \varphi\big(W^{(\ell)} h^{(\ell-1)} + b^{(\ell)}\big), \qquad h^{(0)} = \widetilde{x}. \tag{2.5}$$

Сеть с одним скрытым слоем и линейным выходом задаёт

$$\widehat{z} = v^{\mathsf{T}} \varphi(W\widetilde{x} + b) + c. \tag{2.6}$$

Реализация последовательности таких слоёв соответствует `keras.Sequential`; полносвязный слой представлен `Dense` [11, 12].

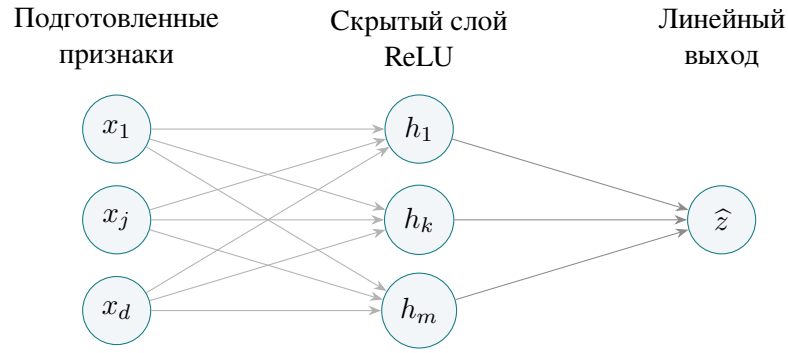


Рис. 1: Схема полносвязной сети. Показаны отдельные компоненты; фактическое число входов определяется после предобработки.

Входная размерность d равна числу подготовленных признаков. Её нельзя произвольно менять при фиксированных данных. В задании об изменении числа нейронов варьируется ширина **первого скрытого полносвязного слоя**. Выходной слой содержит один нейрон, поскольку для каждого дома предсказывается одно число.

У слоя с d входами и m нейронами имеется $dm+m$ обучаемых параметров. Для схемы $d \to m \to 1$ их всего

$$P = (d+1)m + (m+1). \tag{2.7}$$

Например, при $d = 10$, $m = 8$ получаем $P = 97$. Это вычислительная иллюстрация, а не характеристика файлов курса. Сравните собственный расчёт с `model.count_params()`.

2.5. Функции активации

Нелинейная активация позволяет сети представлять нелинейные зависимости. Последовательность только линейных слоёв без активаций сводится к одному аффинному преобразованию — взвешенной сумме входов с добавлением смещения.

Активация	Формула	Особенность
ReLU	$\max(0, a)$	Простая исходная активация скрытых слоёв; на отрицательной полуоси производная равна нулю.
Sigmoid	$1/(1+e^{-a})$	Выход в $(0, 1)$; в областях насыщения производная мала.
Tanh	$\tanh(a)$	Выход в $(-1, 1)$; также имеет области насыщения.
Линейная	a	Не ограничивает диапазон прогноза.

В основной конфигурации используем ReLU внутри сети и линейную функцию на выходе [13]. Sigmoid на выходе без согласованного преобразования цели ограничила бы прогноз интервалом $(0, 1)$. Линейный выход для стандартизованной цели может принимать отрицательные значения: это означает цену ниже μ_y, а не обязательно отрицательную цену. Проверять допустимость нужно после обратного преобразования.

2.6. Функции потерь и метрики регрессии

Пусть на оцениваемой части имеется n объектов, а ошибка прогноза $e_i = y_i - \widehat{y}_i$. Определим

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n} e_i^2, \tag{2.8}$$

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n} |e_i|, \tag{2.9}$$

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n} e_i^2}. \tag{2.10}$$

MAE и RMSE измеряются в единицах цены, MSE — в квадрате этих единиц. Квадратичная ошибка сильнее выделяет большие промахи. MAE придаёт линейный вес абсолютной величине ошибки [14].

> **Пример расчёта**
>
> Пусть цены равны $(100, 150, 200)$, а прогнозы — $(110, 140, 230)$ в условных денежных единицах. Тогда $\mathrm{MAE} = 50/3 \approx 16{,}67$, $\mathrm{MSE} = 1100/3 \approx 366{,}67$, $\mathrm{RMSE} \approx 19{,}15$. Числа служат только для проверки понимания формул.

Дополнительно можно вычислить коэффициент детерминации

$$R^2 = 1 - \frac{\sum_i (y_i - \widehat{y}_i)^2}{\sum_i (y_i - \overline{y})^2}. \tag{2.11}$$

Здесь $\overline{y}$ — среднее по *оцениваемой* части данных в определении метрики, а не параметр предобработки. Значение R^2 может быть отрицательным; это не процент точных ответов. При постоянной цели знаменатель равен нулю, и такая формула не определена [15].

Функция потерь задаёт то, что минимизирует обучение. **Метрика** нужна для оценки и сравнения. При смене MSE на MAE нельзя сравнивать сами значения `loss` как одну общую шкалу. Во всех опытах сравнивайте одинаковые MAE и RMSE после возврата к исходным ценам. Главным критерием выбора в этой работе назначается MAE валидации; RMSE помогает увидеть крупные ошибки.

Для преобразования (2.3) справедливо $\mathrm{MAE}_y = s_y\,\mathrm{MAE}_z$ и $\mathrm{MSE}_y = s_y^2\,\mathrm{MSE}_z$. Поэтому метрика в журнале обучения на стандартизованной цели и ошибка в денежных единицах имеют разные численные значения.

2.7. Обучение: градиент, эпоха и мини-выборка

Обучение изменяет веса так, чтобы уменьшить среднюю потерю. Градиент показывает, в какую сторону потеря растёт быстрее при малом изменении параметров. Шаг в противоположную сторону помогает её уменьшить; величину шага задаёт скорость обучения. Для мини-выборки B_t шаг обычного градиентного метода имеет вид

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \frac{1}{|B_t|} \sum_{i \in B_t} \ell(y_i, f_\theta(x_i)), \tag{2.12}$$

где $\eta > 0$ — скорость обучения. Производные вычисляются с использованием цепного правила; последовательное распространение производных от выхода сети к её входным слоям называют обратным распространением ошибки.

Один шаг обучения на простом примере

Пусть $\widehat{y} = wx + b$, $x = 2$, правильный ответ $y = 6$, начальные $w = 1$, $b = 0$. Прогноз равен 2, а квадратичная потеря $(\widehat{y} - y)^2 = 16$. Производные:

$$\frac{\partial L}{\partial w} = 2(\widehat{y} - y)x = -16, \qquad \frac{\partial L}{\partial b} = 2(\widehat{y} - y) = -8.$$

При шаге $0{,}01$ получаем $w = 1{,}16$, $b = 0{,}08$. Новый прогноз равен $2{,}4$, потеря — $12{,}96$: на этом шаге она уменьшилась. В многослойной сети производные передаются через слои по цепному правилу. Это учебный расчёт для одного объекта, не результат обучения на домах.

Итерация в рассматриваемом режиме — один шаг обновления весов по мини-выборке. **Эпоха** — проход по всей обучающей части. Если в ней N строк и последний неполный пакет сохраняется, число обновлений за эпоху равно $\lceil N/b \rceil$, где b — `batch_size`. При $N = 1000$, $b = 32$ это 32 обновления, а за 10 эпох — 320 [16].

Меньший пакет означает больше обновлений за эпоху и обычно более шумную оценку градиента. Поэтому сравнение пакетов при одинаковом числе эпох не является сравнением

при одинаковом числе обновлений. Слишком большая скорость обучения может привести к колебаниям или расходимости; слишком малая — к медленному улучшению.

В работе сравниваются SGD и Adam. SGD использует градиент текущего пакета; Adam дополнительно накапливает оценки первого и второго моментов градиента и адаптирует масштаб обновлений [17, 18]. Одинаковая численная скорость обучения не означает одинаковые траектории или одинаковую оптимальность настройки для двух алгоритмов.

2.8. Переобучение и воспроизводимость

Регуляризация ограничивает подгонку модели к особенностям обучения. Например, штраф L2 добавляет к потере величину, растущую с квадратами весов. Dropout во время обучения случайно отключает часть выходов скрытого слоя, а при прогнозировании отключение не выполняется. Эти приёмы меняют обучение и требуют проверки на валидации; в базовую конфигурацию они не включены.

Переобучение можно заподозрить, если ошибка на обучении продолжает уменьшаться, а на валидации перестаёт улучшаться или растёт. По одной кривой нельзя автоматически установить причину: проверьте подготовку данных, размер выборки, архитектуру и скорость обучения.

Дополнительный приём — ранний останов: прекратить обучение после заданного числа эпох без улучшения валидационной метрики и вернуть лучшие сохранённые веса. В Keras это реализует `EarlyStopping` [19]. В основных опытах с числом эпох ранний останов выключен, чтобы сравнивать результаты после заданного числа проходов. Его эффект исследуется отдельно.

Используйте одинаковое разбиение и исходный seed 40. Начальные состояния генераторов задаются командой `keras.utils.set_random_seed`. В примере дополнительно включаются детерминированные операции, что уменьшает дополнительную случайность вычислений [20]. Записывайте версии библиотек и устройство. Один seed не является оценкой устойчивости и не гарантирует побитового совпадения на разных программных и аппаратных платформах [21].

2.9. Порядок выполнения

Первый проход: проследите путь одного дома от строки таблицы через подготовку признаков до прогноза цены. Выполните B0 и объясните MAE в денежных единицах. К сравнению десяти конфигураций переходите после проверки этого запуска.

Шаг 1. Изучите исходные таблицы

Откройте обе таблицы и описание данных. Проверьте число строк и столбцов, наличие `SalePrice` только в обучающем файле и уникальность `Id` внутри каждого файла. Назовите два числовых и два категориальных признака; объясните их смысл.

```python
from pathlib import Path
import pandas as pd

DATA_DIR = Path("data")
train_data = pd.read_csv(DATA_DIR / "train.csv")
test_data = pd.read_csv(DATA_DIR / "test.csv")
print(train_data.shape, test_data.shape)
print(train_data.head())
print(test_data.head())
print(train_data.isna().sum().sort_values(ascending=False).head(10))
```

> **Контрольная точка 1**
>
> В `train.csv` цена известна, в `test.csv` — отсутствует. `Id` нужен для связи прогноза с домом и не включается в признаки. Пропущенную целевую цену нельзя заполнять медианой: такая строка не даёт достоверного обучающего ответа.

Шаг 2. Выполните разделение и предобработку

Сохраните фиксированное удаление пяти столбцов из исходного задания. Отделите `SalePrice` и `Id`, затем разделите сырые строки в отношении 80/20 с `random_state=40`. Только теперь обучайте преобразования. Готовая реализация находится в `lab2_utils.py`. Прочитайте функцию `prepare_frames` и найдите в ней место, где вычисляются обучающие статистики.

```python
import sys
sys.path.insert(0, "code")
from lab2_utils import load_course_data, constant_baselines

data = load_course_data(DATA_DIR, seed=40)
print(data["Atr"].shape, data["Ava"].shape, data["Atest"].shape)
print(data["removed_columns"])
print(data["feature_names"][:10])
```

Числовой конвейер объединяет заполнение медианой и стандартизацию. Категориальный — заполнение отдельной категорией и one-hot-кодирование. `ColumnTransformer` применяет эти конвейеры к соответствующим столбцам [22]. Признаки с целочисленными кодами проверьте по словарю: тип хранения сам по себе не доказывает, что признак количественный.

> **Контрольная точка 2**
>
> Три матрицы имеют одинаковое число столбцов и не содержат NaN или бесконечностей. Пересечение `train_ids` и `validation_ids` пусто. Объекты валидации не участвовали в `fit` преобразований. Число входов сети равно `data["Atr"].shape[1]`.

Шаг 3. Рассчитайте простые базовые прогнозы

Для сравнения используйте константные прогнозы: среднюю и медианную цену *обучающей* части. Для каждого вычислите MAE и RMSE на валидации.

```
baseline_table = pd.DataFrame(constant_baselines(data)).T
print(baseline_table)
```

Такая проверка показывает, даёт ли сложная модель пользу по сравнению с решением, не использующим признаки. Нейросеть не обязана выигрывать: если это не произошло, разберите причины и честно покажите результат.

Шаг 4. Постройте исходную нейросеть

Создайте и проанализируйте последовательную модель. Исходная настройка: один скрытый слой из 64 нейронов с ReLU, один линейный выход, Adam, скорость обучения 10^{-3}, MSE. Входная размерность определяется результатом предобработки.

```
from tensorflow import keras

keras.utils.set_random_seed(40)
d = data["Atr"].shape[1]
model = keras.Sequential([
    keras.Input(shape=(d,)),
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dense(1, activation="linear"),
])
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss="mse",
    metrics=[keras.metrics.MeanAbsoluteError(name="mae")],
)
model.summary()
```

Этот небольшой пример показывает работу API. Основное самостоятельное задание — изменить архитектуру, провести предусмотренные исследования и объяснить результаты. Для своей реализации рассчитайте число параметров вручную и проверьте его по сводке модели.

Шаг 5. Обучите модель и сохраните историю

Передавайте в модель `Atr` и стандартизованную цель `ztr`, а валидационные данные задавайте явно. Не используйте переменную `test_data` в аргументе `validation_data`.

```
history = model.fit(
    data["Atr"], data["ztr"],
    validation_data=(data["Ava"], data["zva"]),
    epochs=100, batch_size=32, shuffle=True, verbose=0,
)
```

Сопровождающая функция `run_experiment` объединяет создание новой модели, обучение, обратное преобразование прогноза и расчёт метрик. Используйте либо собственную последовательность шагов, либо эту функцию; не обучайте две модели подряд по ошибке, принимая их за один опыт.

```
from lab2_utils import run_experiment, plot_diagnostics

result = run_experiment(
    data, hidden=(64,), epochs=100, batch_size=32,
    optimizer="adam", learning_rate=1e-3,
    loss="mse", seed=40, early_stop=False,
)
print(result["metrics"])
fig = plot_diagnostics(data, result)
```

Шаг 6. Оцените качество и исследуйте ошибки

Постройте три графика:

1. MAE обучения и валидации в зависимости от эпохи.
2. Предсказанная цена против фактической на валидации; добавьте прямую $\widehat{y} = y$, обозначающую идеальный прогноз.
3. Остаток $e = y - \widehat{y}$ в зависимости от предсказанной цены; добавьте горизонтальную линию $e = 0$.

Во всех подписях укажите используемые единицы. По последним двум графикам обсудите крупные промахи, систематическое завышение или занижение цены, а также изменение разброса ошибки.

Выделите пять объектов с наибольшей абсолютной ошибкой и рассмотрите их исходные характеристики. Не удаляйте их из валидации ради улучшения метрики. Если предполагаете ошибку данных, сформулируйте проверяемую гипотезу и укажите, каких сведений не хватает.

> **Контрольная точка 3**
>
> MAE и RMSE вычислены после обратного преобразования цели. Для каждой конфигурации использованы те же валидационные объекты. У каждой кривой указано, к какой модели и части данных она относится.

Шаг 7. Получите и сохраните прогноз

После выбора конфигурации используйте её для всех строк `test.csv`. В базовом варианте сохраняется модель, обученная на 80% размеченных строк. Её прогнозы сопровождаются оценкой, полученной на оставшихся 20%.

```python
import numpy as np
from lab2_utils import save_experiment

preds = np.asarray(result["predictions"]).reshape(-1)
output = pd.DataFrame({
    "Id": data["prediction_ids"],
    "SalePrice": preds,
})
assert len(output) == len(test_data)
assert output["Id"].tolist() == test_data["Id"].tolist()
assert np.isfinite(output["SalePrice"]).all()
print("Negative prices:", int((output["SalePrice"] < 0).sum()))
save_experiment(data, result, "runs/baseline_01")
```

Не обрезайте отрицательные прогнозы незаметно. Сначала проверьте обучение и обратное преобразование. Если выбирается правило постобработки, обоснуйте его по валидации и одинаково примените к обоим наборам прогнозов. Файл `predictions_raw.csv` сохраняет исходные значения.

Допускается отдельное финальное переобучение на всём `train.csv` после фиксации настроек. В этом случае нужно заново обучить предобработку и сеть. Прежняя валидационная ошибка относится к прежней модели: вычислять её на строках, включённых в финальное обучение, как независимую оценку нельзя.

2.10. Выполнение на PyTorch

Вы можете выполнить работу на PyTorch, сохранив те же разбиение, предобработку, архитектуру, метрики и исследуемые факторы. В обычном интерфейсе PyTorch обучение не выполняется вызовами `model.compile()` и `model.fit()`: цикл записывается явно. Минимальная последовательность действий внутри цикла [23]:

```python
model.train()
for xb, yb in train_loader:
    optimizer.zero_grad()
    prediction = model(xb)
    loss = criterion(prediction, yb)
    loss.backward()
    optimizer.step()

model.eval()
with torch.no_grad():
    val_prediction = model(X_val_tensor)
```

В этом фрагменте загрузчик, модель, оптимизатор и тензоры требуется создать самостоятельно. Проверьте, что прогноз и цель имеют одинаковую форму $(b, 1)$. Сочетание $(b, 1)$ и $(b,)$ может привести к нежелательному расширению размерностей при вычислении потерь. В качестве функций потерь используйте `MSELoss` и `L1Loss`. Результаты TensorFlow

и PyTorch не обязаны совпадать при одинаковом seed: сравнивайте соблюдение протокола и полученные метрики, а не побитовое совпадение весов.

2.11. Самостоятельное исследование гиперпараметров

Перед запуском сформулируйте ожидаемое влияние каждого изменения. Сравнивайте варианты с исходной конфигурацией, меняя один фактор за раз. Для каждого запуска создавайте новую сеть и новый оптимизатор. Продолжение обучения старой сети не является независимым опытом.

> **Фиксированные условия**
>
> Разделение 80/20; seed 40; одинаковая подготовка признаков и цели; ReLU в скрытых слоях; линейный выход; скорость обучения 10^{-3}. Основные опыты проводятся без раннего останова.

Таблица задаёт компактный план из десяти конфигураций. Это предложенный учебный протокол, а не результаты подбора оптимальной модели. Преподаватель может скорректировать бюджет до начала занятия.

Таблица 5: План сравнения. В столбце «слои» указаны ширины скрытых слоёв.

Опыт	Слои	Эпохи	Пакет	Оптимизатор	Потеря
B0	64	100	32	Adam	MSE
W1	32	100	32	Adam	MSE
W2	128	100	32	Adam	MSE
D1	64, 32	100	32	Adam	MSE
E1	64	30	32	Adam	MSE
E2	64	200	32	Adam	MSE
P1	64	100	16	Adam	MSE
P2	64	100	64	Adam	MSE
O1	64	100	32	SGD	MSE
L1	64	100	32	Adam	MAE

1. **Ширина слоя.** Сравните B0, W1 и W2. Как изменились число параметров, ошибка и разрыв между обучением и валидацией?
2. **Глубина.** Сравните B0 и D1. Объясните, какие изменения вносит второй скрытый слой и оправдалось ли усложнение.
3. **Число эпох.** Сравните E1, B0 и E2. Покажите кривые и определите, продолжалось ли улучшение к концу обучения.
4. **Размер мини-выборки.** Сравните P1, B0 и P2. Рассчитайте число обновлений весов для каждого варианта и объясните, почему одинаковое число эпох не означает одинаковый вычислительный опыт.

5. **Оптимизатор.** Сравните B0 и O1 при фиксированном шаге. Вывод ограничьте этими настройками: он не доказывает общего превосходства Adam или SGD при индивидуально подобранных скоростях обучения.
6. **Функция потерь.** Сравните B0 и L1 по одинаковым MAE и RMSE в единицах цены. Разберите изменение крупных ошибок.

Журнал результатов

Для каждого опыта сохраните идентификатор, архитектуру, число параметров, заданные и выполненные эпохи, размер пакета, оптимизатор, скорость обучения, функцию потерь, seed, время обучения, MAE, RMSE и при необходимости R^2. Пример строки ниже содержит названия полей, а не вымышленные результаты.

```
row = {
    "experiment": "B0",
    **result["config"],
    **result["metrics"],
}
pd.DataFrame([row]).to_csv("experiment_B0.csv", index=False)
```

Как выбирать конфигурацию

Сначала сравните MAE валидации, затем рассмотрите RMSE, сложность и диагностические графики. Если разница мала, не объявляйте её устойчивым преимуществом по одному запуску. Зафиксируйте, что обнаружено на данном разбиении, и какая дополнительная проверка нужна.

Гиперпараметры не подбираются по `test.csv`: в нём нет правильных ответов. В этой работе нет заданного универсального порога MAE или требования обязательно превзойти все другие алгоритмы. Оценивается корректность эксперимента и обоснованность выводов.

Дополнительные исследования

По согласованию с преподавателем выполните одно расширение:

- включите ранний останов по MAE валидации и сравните с B0; укажите фактическую длину обучения и правило возврата весов;
- исследуйте влияние отдельной скорости обучения у SGD;
- сравните фиксированное удаление пяти признаков с сохранением их содержательно интерпретированных категорий;
- повторите две выбранные конфигурации с несколькими заранее заданными seed и покажите разброс на том же фиксированном разбиении.

Не смешивайте дополнительные опыты с исходной таблицей без указания, какие условия изменились.

2.12. Типичные ошибки и способы проверки

Наблюдение	Возможная причина	Проверка
Разное число входных признаков	Кодировщик обучен отдельно на каждой таблице	Один `fit` на обучении; одинаковые имена и порядок столбцов.
Подозрительно малая ошибка	Целевая цена попала в признаки или статистики получены до разделения	Проверить список входов и место вызова `fit`.
Ошибка обучения равна NaN	Остались NaN, бесконечности или обучение расходится	Проверить массивы и цель; затем исследовать масштаб и шаг.
Прогнозы около нуля	Забыто обратное преобразование стандартизованной цели	Применить тот же `target_scaler`, который обучен на y_{train}.
Плохое качество у дорогих домов	Большие ошибки на части диапазона, редкие сочетания признаков	Изучить остатки и крупнейшие промахи; не удалять их из валидации ради метрики.
Отрицательные исходные цены	Ошибочная шкала или неподходящее поведение модели	Проверить обратное преобразование; обосновать изменение модели или постобработку на валидации.
Результат меняется при повторе	Другое разбиение, случайная инициализация, продолжение обучения	Сопоставить seed, версии, идентификаторы строк и создание новой модели.
Ошибка размера тензоров в PyTorch	Цель имеет форму $(b,)$, выход — $(b, 1)$	Явно привести обе формы к $(b, 1)$.

2.13. Отчёт и защита работы № 2

Содержание отчёта

1. ФИО, ИТМО ID, название и номер работы.
2. Цель, характеристика исходных данных и смысл используемых признаков.
3. Описание разбиения и предобработки: что удалено, как заполнены пропуски, как обработаны категории и где вычислены статистики.
4. Архитектура исходной сети, размерности и расчёт числа параметров.
5. Таблица базовых прогнозов и всех выполненных конфигураций.
6. Кривые обучения, сравнение фактических и предсказанных цен, анализ остатков и наиболее крупных ошибок.
7. Вывод о влиянии исследованных факторов и обоснование выбранной конфигурации с указанием ограничений.
8. Пример итоговой таблицы прогнозов и описание файлов результата.
9. Список источников и параметры воспроизводимости.

К отчёту приложите ноутбук с выполненными ячейками, исходный код изменений, журнал опытов, истории обучения и прогнозы. Сохраняйте результаты всех запланированных запусков, включая неудачные, с объяснением причины сбоя. Полные CSV не требуется вставлять в PDF отчёта.

Критерии оценки

Критерий	Что необходимо показать
Корректность данных	Разделение до обучения преобразований, нет `Id` и цели во входах, согласованное кодирование.
Корректность модели	Подходящие вход и выход, согласованная потеря, понимание количества параметров и процесса обучения.
Качество исследования	Сопоставимые опыты, сохранённые условия, одинаковые метрики, объяснение результатов.
Анализ и вывод	Разбор ошибок и ограничений; вывод подтверждён собственными наблюдениями.
Воспроизводимость	Данные, версии, разбиение, конфигурации и материалы позволяют повторить опыт.

Баллы, пороги и правила повторной сдачи устанавливает преподаватель по рабочей программе дисциплины. Исправный запуск кода без анализа не демонстрирует всех результатов обучения этой работы.

Контрольные вопросы

1. Почему прогнозирование `SalePrice` относится к регрессии? Чем признак отличается от класса и целевой переменной?
2. Для чего нужны обучение, валидация и `test.csv`? На какой части допустим подбор гиперпараметров?
3. Почему нельзя отдельно обучать кодировщики категорий на обучающих и новых данных? Приведите пример ошибки соответствия кодов.
4. Зачем масштабировать признаки и целевую цену? Как вернуть прогноз к исходным единицам?
5. Что вычисляет полносвязный слой? Как найти число его параметров?
6. Почему входная размерность не меняется вместе с шириной скрытого слоя?
7. Что такое функция активации? Сравните ReLU, sigmoid и tanh. Почему выход данной сети линейный?
8. Что такое эпоха и итерация? Сколько обновлений весов было в вашем опыте и как на это повлиял `batch_size`?
9. Что измеряют MSE, MAE и RMSE? В каких единицах они выражаются?
10. Почему при смене функции потерь нужно сохранять общие метрики сравнения?
11. Как ширина, глубина, число эпох, размер пакета и оптимизатор повлияли на ваши результаты? Какие выводы поддерживаются данными?
12. Какие признаки переобучения вы наблюдали? Как работает ранний останов?
13. Почему лучший результат на одной валидации не гарантирует лучшего результата на будущих домах?
14. Можно ли судить о качестве прогноза по первым строкам таблицы `Id, SalePrice`, если правильная цена неизвестна?

> **Перед сдачей**
>
> Убедитесь, что ноутбук выполняется последовательно из чистого состояния, в отчёте есть собственные результаты всех обязательных сравнений, а файл прогнозов содержит ровно те `Id` и в том же порядке, что исходный `test.csv`.

Практическая работа № 3. Свёрточные нейронные сети

3.1. Постановка задачи и результат работы

Требуется классифицировать изображения пяти видов цветов: ромашки, одуванчики, розы, подсолнухи и тюльпаны. Исходное задание использует набор Flowers Recognition. Изображения помещаются в каталоги `daisy`, `dandelion`, `rose`, `sunflower`, `tulip`; число файлов и соответствие индексов меткам нужно получить из своей копии данных.

Обязательное сравнение сохраняет три части исходника:

1. заданная свёрточная сеть, обучаемая с нуля;
2. предобученная свёрточная архитектура с новой головой классификации;
3. собственная архитектура студента, обучаемая с нуля.

Необходимо объяснить устройство моделей, визуализировать результаты и сравнить качество при одинаковом разбиении. В пособии используется PyTorch, как и в коде исходного ноутбука. При выполнении на TensorFlow/Keras нужно явно перенести слои, режимы, предобработку и протокол, а не смешивать API двух фреймворков.

Результат сдачи: схема каждой архитектуры, число всех и обучаемых параметров, кривые обучения, таблица сравнения трёх моделей, матрицы ошибок, примеры прогнозов, сохранённые веса и описание воспроизводимого запуска.

3.2. Изображение как тензор

Тензор здесь — многомерный массив чисел. Цветное изображение содержит три канала RGB: красный, зелёный и синий. В PyTorch пакет изображений имеет форму (B, C, H, W): размер пакета, число каналов, высота, ширина. Для принятого входа это $(B, 3, 224, 224)$. `ImageFolder` читает метки из имён каталогов и формирует `class_to_idx`; используйте это отображение и при обучении, и при выводе названий [24].

Преобразование в тензор переводит обычные 8-битные RGB-пиксели в значения от 0 до 1. Затем выполняется нормализация каналов:

$$z_{c,h,w} = \frac{x_{c,h,w} - \mu_c}{\sigma_c}, \qquad x_{c,h,w} = \sigma_c z_{c,h,w} + \mu_c. \tag{3.1}$$

В исходном примере используются $\mu = (0{,}485, 0{,}456, 0{,}406)$ и $\sigma = (0{,}229, 0{,}224, 0{,}225)$. Это фиксированные константы нормализации ImageNet, а не статистики, вычисленные по цветам. Для визуализации сначала выполните обратное преобразование; прямой `clip(0,1)` нормализованного тензора искажает цвета.

В исходном сравнении все модели получают `Resize((224,224))`, преобразование в тензор и одну нормализацию. Такое изменение размера может менять пропорции изображения. У весов ResNet18 есть собственный рекомендованный рецепт `weights.transforms()`: изменение размера и центральная обрезка [25]. Это другой рецепт. Если сравниваете его с исходным, оформите отдельный опыт и применяйте выбранные правила одинаково к сравниваемым моделям.

Аугментация создаёт изменённые варианты обучающих изображений, например зеркальные отражения. Она помогает показать модели больше допустимых вариантов объекта, но преобразование должно сохранять его класс. В основном сравнении этой работы случайные преобразования выключены; их влияние можно исследовать отдельно. На валидации применяются фиксированные преобразования.

3.3. Свёрточный слой

Полносвязный слой рассматривает все входные координаты совместно. Свёрточный слой использует небольшой локальный участок и одни и те же веса в разных пространственных положениях. Для шага 1 и без дополнения границы один выходной канал вычисляется по правилу

$$a_{o,h,w} = b_o + \sum_{c=1}^{C_{in}} \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} W_{o,c,u,v}\, x_{c,h+u,w+v}. \tag{3.2}$$

В `Conv2d` реализуется взаимная корреляция: ядро в этой записи не переворачивается. В терминологии нейронных сетей операцию традиционно называют свёрткой [26]. Полученные карты признаков — таблицы откликов фильтров в разных участках изображения — проходят через нелинейность, например ReLU. Обучение определяет, какие локальные сочетания полезны для классификации.

> **Маленький расчёт**
>
> Для одной карты $X = \left[\begin{smallmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{smallmatrix}\right]$, ядра $K = \left[\begin{smallmatrix} 1 & 0 \\ 0 & 2 \end{smallmatrix}\right]$, нулевого смещения и шага 1 без padding получается $\left[\begin{smallmatrix} 11 & 14 \\ 20 & 23 \end{smallmatrix}\right]$. Например, левый верхний элемент равен $1 + 2 \cdot 5 = 11$. Найдите остальные элементы вручную.

Для размера входа H, ядра k, шага s, padding p и dilation d высота выхода равна

$$H_{out} = \left\lfloor \frac{H + 2p - d(k-1) - 1}{s} + 1 \right\rfloor. \tag{3.3}$$

Для ширины действует такая же формула. В этой работе $d = 1$. При $k = 3$, $s = 1$, $p = 1$ размер сохраняется. Для обычной свёртки без групп со смещением число параметров

$$P = C_{out}(C_{in} k_h k_w + 1). \tag{3.4}$$

Первый слой $3 \to 32$ с ядром 3×3 содержит $32(3 \cdot 3 \cdot 3 + 1) = 896$ параметров. Общие веса резко уменьшают число параметров относительно полносвязного соединения всех пикселей со всеми пространственными выходами.

3.4. Pooling и устройство исходной сети

`MaxPool2d(2,2)` выбирает максимум в каждом окне 2×2 и уменьшает высоту и ширину вдвое для чётного входа. Он не обучает веса. Pooling уменьшает пространственную

детализацию и стоимость следующих вычислений; это не означает полной устойчивости классификатора к любому сдвигу или повороту.

Таблица 7: Размерности исходной CNN при входе 224×224

Последовательность операций	Форма без размера пакета
Вход RGB	$3 \times 224 \times 224$
Conv $3 \to 32$, ReLU, Conv $32 \to 64$, ReLU, MaxPool	$64 \times 112 \times 112$
Conv $64 \to 128$, ReLU, Conv $128 \to 128$, ReLU, MaxPool	$128 \times 56 \times 56$
Conv $128 \to 256$, ReLU, Conv $256 \to 256$, ReLU, MaxPool	$256 \times 28 \times 28$
Flatten	200704
Linear, ReLU; Linear, ReLU; Linear	$1024 \to 512 \to 5$

Все свёртки в таблице имеют ядро 3, шаг 1 и padding 1. Полная реализация `original_cnn()` в учебном модуле сохраняет размеры слоёв полученного ноутбука. Flatten только меняет представление тензора: он не извлекает новые признаки.

> **Почему нужно заранее оценить память**
>
> Слой $200704 \to 1024$ содержит $205\,521\,920$ параметров со смещением. Вся исходная сеть — $207\,175\,365$. Только веса float32 занимают около 0,77 GiB. Веса, градиенты и два состояния Adam суммарно требуют около 3,09 GiB, ещё без активаций и временных буферов. Это расчёт по архитектуре, а не измерение пикового расхода памяти.

Для собственной модели полезно исследовать глобальное усреднение карт. В примере оно выполняется слоем `AdaptiveAvgPool2d(1)`. Этот слой превращает $C \times H \times W$ в $C \times 1 \times 1$, после чего компактная голова предсказывает пять классов. Это уменьшает число параметров, но меняет модель и не гарантирует лучшего качества. В комплекте дан небольшой пример; самостоятельная архитектура должна содержать обоснованное собственное изменение.

3.5. Логиты, cross-entropy и обучение

Последний слой выдаёт пять логитов $z_1, \ldots, z_5$ — оценок классов, которые могут быть любыми вещественными числами. Softmax превращает их в положительные числа с суммой 1. Вероятности softmax определяются как

$$p_c = \frac{e^{z_c}}{\sum_{j=1}^{5} e^{z_j}}, \qquad L = -\frac{1}{B} \sum_{i=1}^{B} \log p_{i,y_i}. \tag{3.5}$$

`CrossEntropyLoss` принимает исходные логиты формы $(B, 5)$ и целочисленные индексы классов формы $(B,)$ с типом `torch.long`. Отдельный Softmax перед этой функцией не нужен [27]. Для выбора класса достаточно `logits.argmax(dim=1)`; наибольший логит и наибольшая вероятность имеют один индекс. Вероятность softmax сама по себе не является проверенной уверенностью модели.

Итерация обучения включает обнуление градиентов, прямой проход, вычисление потери, обратный проход и шаг оптимизатора. При проверке обратный проход и шаг отсутствуют. `model.eval()` переключает поведение слоёв, а отключение градиентов снижает затраты на вычислительный граф: это разные операции.

Метрики считают заново для каждой эпохи и каждой фазы. Средняя потеря должна учитывать размер последнего пакета:

$$L_{epoch} = \frac{\sum_b n_b L_b}{\sum_b n_b}. \tag{3.6}$$

Например, для пакетов из 32 и 8 изображений нельзя брать простое среднее двух потерь. Accuracy считают по всем правильным ответам фазы, macro-F1 — по всем её меткам и прогнозам, а не как невзвешенное среднее F1 пакетов.

3.6. Перенос обучения и заморозка

Предобученная сеть уже получила веса на другой задаче. Основную часть сети, преобразующую изображение в признаки, называют **извлекателем признаков (backbone)**. Завершающий блок, переводящий эти признаки в ответы нашей задачи, называют **головой классификации (head)**. В переносе обучения её признаки применяются к новому набору. В исходном варианте оставляют фиксированный извлекатель признаков и обучают новую голову; при **дообучении (fine-tuning)** дополнительно изменяют часть предобученных слоёв. Эти режимы нужно различать в отчёте [28].

Пример использует ResNet18 с явно указанными весами `IMAGENET1K_V1`. Основой сети служат остаточные блоки: к преобразованию входа добавляется путь, передающий сам вход. Если размеры совпадают, блок имеет вид $h_{next} = \mathrm{ReLU}(F(h) + h)$; при изменении размеров применяется согласующее преобразование короткого пути. Четыре группы блоков используют 64, 128, 256 и 512 каналов; после глобального усреднения заменяется слой `fc` на $512 \to 5$ [29, 25].

`requires_grad=False` исключает параметры из расчёта их градиентов. Новая голова создаётся после заморозки и остаётся обучаемой; её параметры включаются в оптимизатор. BatchNorm нормализует промежуточные значения в сети. У него есть также текущие средние и дисперсии — сохраняемые служебные значения (буферы), изменяющиеся в режиме обучения [30]. Для полностью фиксированного извлекателя в данном протоколе backbone работает в `eval`, а новая голова — в `train`. Простого общего вызова `model.train()` здесь недостаточно.

Сравнение с обучением с нуля учитывает дополнительный ресурс: веса ResNet получены на внешних данных. Поэтому вывод относится к практическим стратегиям обучения при указанном предобучении; это не сравнение одной лишь архитектуры при одинаковом объёме исходной информации.

3.7. Подготовка изображений и разбиение

Первый проход: откройте одно изображение, найдите его метку и форму тензора, затем проверьте размер выхода модели. До длительного обучения оцените память и выполните короткий пробный проход. Он проверяет запуск, но ещё не показывает качество обучения.

Шаг 1. Изучите набор и проверьте ресурсы

Выведите пять классов, число изображений каждого класса и несколько исходных примеров. Проверьте читаемость файлов, размеры, цветовые каналы и дубликаты. Не задавайте число изображений по чужому описанию: оно зависит от версии выгрузки. Выберите устройство и запишите его в отчёте.

```
import torch
from lab3_utils import prepare_flowers, seed_everything
from lab3_utils import original_cnn, transfer_resnet18, compact_example
from lab3_utils import fit_flowers, plot_learning, parameter_counts
seed_everything(0)
device = "cuda" if torch.cuda.is_available() else "cpu"
print(device)
```

Шаг 2. Зафиксируйте одну валидацию

Исходный ноутбук выделяет 1000 изображений. В исправленном варианте это ориентир размера валидации; точные копии файлов объединяются по SHA-256 и попадают в одну часть. Поэтому фактическое число проверочных изображений может отличаться от 1000. `split.csv` сохраняет итоговое распределение. Групповое разделение не гарантирует одинаковые пропорции классов: выведите их и проверьте наличие всех пяти классов в обеих частях.

```
data = prepare_flowers("data", validation_target=1000,
                       seed=0, image_size=224, augment=False)
print(data["class_to_idx"])
print(data["inventory"].groupby(["split", "target"]).size())
print(data["config"])
```

Хеши находят побайтовые копии. Снимки одной сцены после повторного сжатия, обрезки или изменения размера могут иметь разные хеши. Если такие серии известны, их также следует объединить до разделения. Совпадающие файлы с разными метками требуют проверки разметки. Не исправляйте конфликты только ради улучшения метрики.

Обучающий и проверочный наборы создаются как отдельные `ImageFolder` с одним списком файлов и разными подмножествами. При дополнительном опыте со случайным отражением оно применяется только к обучению. Замена `transform` у общего объекта под двумя `Subset` изменила бы обе части одновременно.

Контрольная точка 1

Пакет имеет форму $(B, 3, 224, 224)$, метки — целочисленный вектор длины B со значениями от 0 до 4. Известно отображение индексов в имена. Обучение и валидация не пересекаются по файлам и точным дубликатам. Все три архитектуры используют одно сохранённое разбиение.

3.8. Обучение и исследование архитектур

Шаг 3. Разберите и обучите заданную CNN

Выпишите размеры тензоров после свёрток и pooling, проверьте вход первого Linear и число параметров. Создайте новую модель, затем оптимизатор. Функция ниже использует Adam с явно переданным шагом, десять эпох и пакет 32. Её код доступен в `lab3_utils.py`; найдите места сброса метрик, переключения режима и выбора сохранённых весов.

```
seed_everything(0)
model = original_cnn()
print(parameter_counts(model))
result_original = fit_flowers(
    model, data, "runs/lab3_original_01", epochs=10,
    batch_size=32, learning_rate=1e-3, seed=0, device=device
)
```

Перед полным запуском проверьте один пакет. Если памяти недостаточно, уменьшите пакет и заново зафиксируйте общий протокол для всех трёх моделей. Уменьшение разрешения у исходной сети потребует изменения первого Linear; это уже другая архитектура и должно быть указано. Не заносите ошибку памяти в таблицу качества как нулевую accuracy.

Шаг 4. Примените перенос обучения

Выберите предобученную архитектуру, прочитайте её описание и определите слой классификатора. Пример ниже конкретизирует ResNet18. Порядок важен: сначала заморозка старых параметров, затем создание новой головы.

```
from torchvision import models
from torch import nn
weights = models.ResNet18_Weights.IMAGENET1K_V1
model = models.resnet18(weights=weights)
for p in model.parameters():
    p.requires_grad = False
model.fc = nn.Linear(model.fc.in_features, 5)
model.frozen_backbone = True
for name, p in model.named_parameters():
    if p.requires_grad:
        print(name, p.shape)
```

Вывод должен содержать только `fc.weight` и `fc.bias`. Для головы $512 \to 5$ обучается $512 \cdot 5 + 5 = 2565$ параметров. При первом использовании веса могут загружаться из сети; `weights=None` создаёт случайную инициализацию и не является переносом обучения.

Фабрика `transfer_resnet18()` выполняет эти действия и сохраняет обозначение весов в метаданных. Используйте новую модель и новый каталог опыта:

```
seed_everything(0)
model = transfer_resnet18(pretrained=True)
result_transfer = fit_flowers(
    model, data, "runs/lab3_transfer_01", epochs=10,
    batch_size=32, learning_rate=1e-3, seed=0, device=device
)
```

В режиме фиксированного извлекателя проверьте, что параметры backbone и статистики BatchNorm не меняются после шага. Если исследуете fine-tuning, явно укажите размороженные слои, режим BatchNorm и новый оптимизатор. Не представляйте этот дополнительный опыт как тот же режим заморозки.

Шаг 5. Создайте собственную архитектуру

Предложите изменение, связанное с проверяемой гипотезой: уменьшение головы, другая ширина блоков, изменение глубины, регуляризация или иной способ агрегирования карт. Нарисуйте последовательность слоёв, рассчитайте формы и число параметров. Выход обязан иметь пять логитов. Маленькая модель ниже иллюстрирует конструкцию с глобальным усреднением:

```
student_model = nn.Sequential(
    nn.Conv2d(3, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
    nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
    nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(),
    nn.AdaptiveAvgPool2d(1), nn.Flatten(), nn.Linear(64, 5)
)
```

Самостоятельная часть: внесите как минимум одно осмысленное структурное изменение относительно данного примера и объясните ожидаемый эффект. Простое копирование `compact_example()` не считается разработкой своей архитектуры. Обучите свой вариант с тем же разбиением и основным бюджетом. Зафиксируйте seed *до* создания слоёв, а не только перед вызовом функции обучения.

3.9. Сравнение, визуализация и сохранение

Шаг 6. Соберите сопоставимые результаты

Основной протокол: RGB 224×224, фиксированная нормализация, без случайной аугментации, seed 0, Adam с шагом 10^{-3}, пакет 32, десять эпох. Для каждой модели создаются

новые загрузчики с тем же начальным seed. Выбирается эпоха с наибольшим macro-F1 валидации; при равенстве сохраняется более ранняя. Это одинаковое правило выбора, но не одинаковый объём предобучения.

Таблица 8: Что должно быть в итоговой таблице CNN

Группа полей	Содержание
Модель	Имя, схема, число всех и обучаемых параметров; исходные веса
Обучение	Разбиение, seed, эпохи, пакет, оптимизатор, шаг, размер входа, аугментация
Результат	Лучшая эпоха, loss, accuracy и macro-F1 одной валидации
Ресурсы	Устройство, измеренное время с указанием границ измерения; при измерении — пиковая память
Вывод	Какие классы и случаи улучшились или ухудшились; ограничения сравнения

В модуле время включает обучение, валидацию и запись контрольных точек; оно не включает первоначальную загрузку предобученных весов и подготовку набора. Не сравнивайте его с измерением только одного прямого прохода. Одинаковое число эпох также не означает одинаковое число операций для разных сетей.

Шаг 7. Постройте графики и исследуйте прогнозы

Для каждой модели покажите train/validation loss и macro-F1 по эпохам, а затем общую диаграмму результатов трёх архитектур. Сохраните матрицы ошибок и classification report по пяти классам. Используйте одинаковые оси и порядок меток, подпишите доли или проценты.

```
fig = plot_learning(result_transfer)
fig.savefig("runs/lab3_transfer_01/curves.pdf", bbox_inches="tight")
```

Покажите девять заранее определённых изображений валидации либо все, если их меньше: истинный и предсказанный класс, восстановленные цвета. Дополнительно разберите несколько ошибок; не выбирайте только удачные картинки. Фрагмент демонстрирует корректный прогноз одного нормализованного изображения:

```python
from lab3_utils import denormalize
import matplotlib.pyplot as plt
index = int(data["validation_indices"][0])
image, label = data["validation_dataset"][index]
model = result_transfer["model"]
model.eval()
with torch.no_grad():
    logits = model(image.unsqueeze(0).to(device))
predicted = int(logits.argmax(1).item())
plt.imshow(denormalize(image).permute(1, 2, 0))
plt.title(f"True: {data['classes'][label]}; predicted: {data['classes'][predicted]}")
plt.axis("off")
```

Шаг 8. Сохраните воспроизводимый комплект

Каждая архитектура получает свой каталог. Основные файлы:

- `split.csv` — список изображений и разбиение;
- `history.csv` — история обучения по эпохам;
- `best_weights.pth` — выбранные веса;
- `manifest.json` — параметры и результаты опыта;
- `architecture.txt` — текстовое описание модели.

Также сохраняются прогнозы, отчёт и матрица ошибок. Приложите исходный код собственной архитектуры. Для восстановления нужны совместимое определение модели, те же преобразования и тот же словарь классов, а не только файл весов.

Валидация используется для выбора эпохи и сравнения архитектур. Поэтому её лучший результат нельзя назвать независимой оценкой на новых фотографиях. Дополнительный финальный тест требует отдельного набора, не использованного при этих решениях.

3.10. Контрольные вопросы и критерии готовности

1. Чем CNN отличается от полносвязной сети по связям, параметрам и работе с пространством?
2. Что задают kernel size, stride, padding, входные и выходные каналы? Рассчитайте размер выхода одного слоя.
3. Как работают свёртка и pooling в исходной сети? Какие из них имеют обучаемые параметры?
4. Почему первый Linear исходной архитектуры требует так много памяти?
5. Что такое логиты? Почему перед CrossEntropyLoss не добавляется Softmax?
6. Зачем нужны одновременно eval и отключение градиентов?
7. Что такое предобучение, transfer learning и fine-tuning? Откуда взяты веса выбранной модели?

8. Как устроена выбранная предобученная архитектура? Для ResNet объясните короткий путь и сложение в блоке.
9. Что делает requires_grad? Почему заморозка параметров не равна заморозке статистик BatchNorm?
10. Какое изменение вы внесли в собственную сеть и какую гипотезу проверяли?
11. Почему нельзя накапливать счётчики разных эпох или усреднять F1 пакетов как F1 всего набора?
12. Как корректно показать нормализованное изображение? Почему clip без обратной нормализации неверен?
13. Что можно и чего нельзя заключить из сравнения с предобученной сетью?

> **Перед сдачей работы № 3**
>
> Обучены и сопоставлены все три требуемые архитектуры; объяснены свёртка, pooling, перенос обучения и заморозка. Есть собственное изменение модели, общая таблица, графики и анализ ошибок. Преобразования и разбиение согласованы, метрики относятся к отдельным эпохам, а не ко всему предыдущему обучению.

Общие требования к отчёту и рекомендации преподавателю

Структура отчёта

Для каждой работы укажите ФИО, ИТМО ID, номер и название. Далее приведите цель, описание исходных данных, протокол разбиения, исходную модель, план опытов, результаты и выводы. Полные таблицы данных и многотысячные листинги не требуется вставлять в PDF: они передаются отдельными файлами. В тексте должны оставаться сведения, необходимые для понимания и проверки вывода.

1. **Постановка.** Что предсказывается, какие данные доступны и какая ошибка существенна.
2. **Протокол.** Как получены части данных, что обучалось только на training, по какой метрике выбирались настройки.
3. **Модель.** Принцип работы, архитектура или алгоритм, параметры опыта и мотивация выбора.
4. **Результаты.** Таблица всех запланированных опытов, необходимые графики, разбор ошибок.
5. **Вывод.** Наблюдение, возможное объяснение, ограничение и следующий проверяемый шаг.

Критерии оценивания

Критерий	Наблюдаемый признак выполнения
Данные	Известны признаки и цель; нет утечки, перепутанных выборок и неверных меток.
Модель	Студент объясняет алгоритм, размерности, потери и изменяемые настройки.
Эксперимент	Сравнения воспроизводимы; сохранены разбиения, seed, версии и результаты.
Анализ	Показаны ошибки и ограничения; выводы следуют из полученных результатов.
Самостоятельность	Есть обоснованный выбор, исследование параметров и собственная архитектура в работе № 3.

Баллы, часы, минимальные пороги и правила пересдачи определяются рабочей программой дисциплины. Не следует автоматически оценивать работу только по максимальной метрике: утечка может дать высокий результат при неправильном эксперименте. Напротив, корректно исследованный неудачный вариант может содержать содержательный учебный результат.

Как организовать занятие

До начала курса подготовьте проверенные версии файлов и описаний, уточните названия служебных столбцов, распределите доступные вычислительные ресурсы. Для работы

№ 3 отдельно оцените память исходной архитектуры. После пробного выполнения студентом с заявленной подготовкой уточните трудоёмкость; произвольное количество часов в пособии не назначено.

Полезно проводить короткий допуск: студент объясняет роли файлов, рисует разбиение и вычисляет один пример метрики или размерности. В работе № 1 проверяется логика сравнения, в № 2 — согласованная подготовка признаков и шкала цены, в № 3 — режимы сети и самостоятельность архитектурного решения.

Для первого курса машинного обучения полезно разделить занятие на три прохода: сначала разобрать данные и получить один прогноз, затем объяснить его и метрику, после этого переходить к таблице опытов. На первом проходе не требуется одновременно менять все настройки. Обзор современных табличных моделей и опыт с ансамблем являются дополнительными и не входят в обязательный минимум.

Стартовые ноутбуки содержат рабочие примеры. На защите предложите студенту объяснить конкретную строку, предсказать эффект изменения параметра и сопоставить это ожидание со своим результатом. Не требуйте скрытого эталонного значения accuracy или MAE, которое зависит от версии данных и протокола.

Дополнительные исследования

После обязательного минимума можно исследовать другой способ извлечения признаков активности, содержательную обработку отсутствующих характеристик домов, аугментации изображений или частичный fine-tuning. Каждое расширение должно иметь собственную гипотезу и фиксированные условия; увеличение числа опытов без вопроса не делает исследование более убедительным.

Современные модели для таблиц: дополнительное чтение

От изученной сети к новым подходам

Во второй работе полносвязная сеть обучалась на признаках домов. Её также называют MLP (multilayer perceptron, многослойный перцептрон). Исследователи продолжают развивать такие модели: меняют способ объединения прогнозов и используют предварительное обучение. Ниже рассмотрены две идеи. Их реализация не требуется для выполнения основных работ.

TabM: несколько прогнозов внутри одной модели

Ансамбль объединяет ответы нескольких моделей. Представим три оценки цены одного дома: 180, 190 и 200 условных единиц. Их среднее равно 190. Если ошибки участников различаются, усреднение может уменьшить ошибку; одинаковые систематические промахи таким способом не устраняются.

Обычный ансамбль отдельных нейросетей требует обучить и хранить каждую из них. В **TabM**, представленном на ICLR 2025, несколько вариантов MLP совместно используют большую часть параметров и обучаются вместе. Модель формирует несколько прогнозов для объекта и объединяет их. Это позволяет экономнее реализовать идею ансамбля [32].

Для первого знакомства достаточно различать «одна сеть — один прогноз» и «несколько согласованно обучаемых вариантов — объединённый прогноз». Приведённое выше среднее иллюстрирует объединение ответов, но не воспроизводит устройство TabM.

TabPFN-3: использовать опыт предварительного обучения

Предобученной называют модель, параметры которой настроены заранее. В **TabPFN-3** предварительное обучение выполняется на искусственно созданных таблицах. При решении новой задачи модель получает примеры с известными ответами и признаки объектов, для которых ответ нужно предсказать. Обучающие примеры выступают контекстом для прогноза; это называют *in-context learning* — обучением на примерах в контексте [33].

Здесь важно различать два этапа: затратное предварительное обучение и применение готовой модели к новой таблице. Во втором случае прогноз не обязательно требует обычной настройки весов сети на каждом новом наборе. Модель при этом всё равно использует известные ответы обучающих примеров.

Источник по TabPFN-3 — **технический отчёт мая 2026 года**. Заявления о качестве относятся к экспериментам его авторов. Они не устанавливают качество на файлах нашего курса и не означают, что метод будет лучшим для любой таблицы.

Что сравнивать, кроме названия модели

Новый метод оценивают по тем же вопросам, что и изученную MLP: какие данные он получает, как разделены примеры, какая ошибка измеряется, сколько времени и памяти требуется. Предобучение — дополнительный ресурс, который нужно указать. Улучшение метрики на чужом наборе не заменяет проверку на своей задаче.

Подход	Основная идея
MLP из работы № 2	Настроить одну полносвязную сеть на обучающих домах.
TabM	Совместно обучать несколько вариантов предсказания с общими параметрами.
TabPFN-3	Применить заранее обученную модель, используя примеры новой таблицы как контекст.

Необязательное задание: помогает ли усреднение?

Этот опыт можно выполнить средствами работы № 2, не устанавливая TabM или TabPFN-3. Он знакомит с идеей ансамбля отдельных сетей; это не реализация этих методов.

1. Сохраните одно разбиение домов и одну предобработку из работы № 2. Выберите исходную конфигурацию B0.
2. Обучите три новые копии этой сети с seed 40, 41 и 42. Seed обучения влияет на начальные веса и случайный порядок объектов. Разбиение данных и остальные настройки оставьте теми же.
3. Для каждой сети сохраните прогнозы валидации в исходных единицах цены. В учебном модуле это поле `validation_predictions` результата `run_experiment`.
4. Для каждого дома усредните три прогноза:

$$\widehat{y}_i^{avg} = \frac{\widehat{y}_i^{(1)} + \widehat{y}_i^{(2)} + \widehat{y}_i^{(3)}}{3}.$$

   Усредняются ответы для одного и того же дома, а не значения MAE трёх запусков.
5. Составьте четыре строки результата: три отдельные сети и усреднение. Для каждой укажите MAE и RMSE на одной валидации. Сравните также затраты на три обучения и на одно.

Покажите один дом, для которого усреднение помогло, и один, для которого не помогло, если такие примеры есть. Объясните наблюдение. Укажите, что преимущество на одном разбиении не доказывает общего превосходства, а разброс трёх прогнозов сам по себе не является гарантированным интервалом цены.

Форма сдачи: небольшая таблица, два разобранных примера и вывод на полстраницы. Задание дополнительное и выполняется в среде работы № 2. Для него не требуются библиотеки TabM/TabPFN-3, доступ к их API или их предобученные веса.

Список литературы

[1] Reyes-Ortiz J., Anguita D., Ghio A., Oneto L., Parra X. Human Activity Recognition Using Smartphones. UCI Machine Learning Repository, 2013. https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones.

[2] scikit-learn. GroupShuffleSplit. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GroupShuffleSplit.html.

[3] scikit-learn. Common pitfalls and recommended practices. https://scikit-learn.org/stable/common_pitfalls.html.

[4] scikit-learn. StandardScaler. https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html.

[5] scikit-learn. KNeighborsClassifier. https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html.

[6] scikit-learn. LinearSVC. https://scikit-learn.org/stable/modules/generated/sklearn.svm.LinearSVC.html.

[7] scikit-learn. RandomForestClassifier. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html.

[8] scikit-learn. classification_report. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html.

[9] scikit-learn. SimpleImputer. https://scikit-learn.org/stable/modules/generated/sklearn.impute.SimpleImputer.html.

[10] scikit-learn. OneHotEncoder. https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OneHotEncoder.html.

[11] Keras. The Sequential model. https://keras.io/guides/sequential_model/.

[12] Keras. Dense layer. https://keras.io/api/layers/core_layers/dense/.

[13] Keras. Layer activation functions. https://keras.io/api/layers/activations/.

[14] Keras. Regression losses. https://keras.io/api/losses/regression_losses/.

[15] scikit-learn. Metrics and scoring: quantifying the quality of predictions. https://scikit-learn.org/stable/modules/model_evaluation.html.

[16] Keras. Model training APIs. https://keras.io/api/models/model_training_apis/.

[17] Keras. SGD. https://keras.io/api/optimizers/sgd/.

[18] Keras. Adam. https://keras.io/api/optimizers/adam/.

[19] Keras. EarlyStopping. https://keras.io/api/callbacks/early_stopping/.

[20] Keras. Python & NumPy utilities. https://keras.io/api/utils/python_utils/.

[21] TensorFlow. enable_op_determinism. https://www.tensorflow.org/api_docs/python/tf/config/experimental/enable_op_determinism.

[22] scikit-learn. ColumnTransformer. https://scikit-learn.org/stable/modules/generated/sklearn.compose.ColumnTransformer.html.

[23] PyTorch. Optimizing Model Parameters. https://docs.pytorch.org/tutorials/beginner/basics/optimization_tutorial.html.

[24] Torchvision. ImageFolder. https://docs.pytorch.org/vision/stable/generated/torchvision.datasets.ImageFolder.html.

[25] Torchvision. ResNet18 and pretrained weights. https://docs.pytorch.org/vision/stable/models/generated/torchvision.models.resnet18.html.

[26] PyTorch. Conv2d. https://docs.pytorch.org/docs/stable/generated/torch.nn.Conv2d.html.

[27] PyTorch. CrossEntropyLoss. https://docs.pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html.

[28] PyTorch. Transfer Learning for Computer Vision Tutorial. https://docs.pytorch.org/tutorials/beginner/transfer_learning_tutorial.html.

[29] He K., Zhang X., Ren S., Sun J. Deep Residual Learning for Image Recognition. CVPR, 2016. https://arxiv.org/abs/1512.03385.

[30] PyTorch. BatchNorm2d. https://docs.pytorch.org/docs/stable/generated/torch.nn.BatchNorm2d.html.

[31] Mamaev A. Flowers Recognition. Набор изображений из исходного задания. https://www.kaggle.com/datasets/alxmamaev/flowers-recognition.

[32] Gorishniy Y., Kotelnikov A., Babenko A. TabM: Advancing Tabular Deep Learning with Parameter-Efficient Ensembling. ICLR, 2025. https://arxiv.org/abs/2410.24210.

[33] Grinsztajn L., Flöge K., Key O. et al. TabPFN-3: Technical Report. Технический отчёт, май 2026, версия 2. https://arxiv.org/abs/2605.13986v2.

Дата обращения к документации и наборам данных: 15.09.2026; к статьям TabM и TabPFN-3: 16.09.2026. API библиотек сверялся с официальной документацией; при использовании другой версии проверьте сигнатуры вызываемых функций.

Приложение А. Материалы эксперимента

Каждому запуску назначьте отдельную папку. Ниже приведён пример для работы № 2:

```
runs/B0_01/
    manifest.json
    history.csv
    validation_predictions.csv
    predictions_raw.csv
    model.keras
    preprocessing.joblib
    diagnostics.pdf
```

`manifest.json` фиксирует конфигурацию, метрики валидации, идентификаторы обучающих и валидационных объектов, версии библиотек, контрольные суммы файлов и имена подготовленных признаков. `history.csv` содержит значения по эпохам. `validation_predictions.csv` связывает прогноз с известной ценой для анализа ошибок. Модель и параметры предобработки сохраняются вместе: применение модели к иначе подготовленным признакам некорректно.

Графики добавляются после сохранения опыта:

```
fig.savefig("runs/B0_01/diagnostics.pdf", bbox_inches="tight")
```

Для названий файлов придерживайтесь одного идентификатора опыта: в приведённой строке папка `runs/B0_01` должна совпасть с аргументом вашего вызова `save_experiment`. Не открывайте файлы сериализованной предобработки из неизвестных источников; этот комплект предназначен для сохранённых вами опытов.

Минимальный шаблон вывода

«При фиксированных . . . изменение . . . привело к . . . на валидационной части. Это видно по . . . . Возможное объяснение состоит в . . . . Вывод ограничен . . . . Для проверки устойчивости потребовалось бы . . . ».

Заполните пропуски конкретными условиями, результатами и наблюдениями. Фразы «модель хорошо обучилась» или «стало точнее» без указания метрики и сравнения не объясняют полученный результат.

Что сохраняется в других работах

В работе № 1 сохраняются таблица сравнения валидации, тестовые прогнозы, classification report, обученный Pipeline и описание группового разбиения. В работе № 3 — список изображений с метками и разбиением, история каждой эпохи, выбранные веса, описание архитектуры, словарь классов, метрики и прогнозы.

Для повторения собственной CNN требуется её исходный код. Один файл весов не описывает архитектуру, подготовку изображения или смысл номера класса. Материалы всех трёх работ сопровождаются версиями среды, seed и графиками.

Словарь основных обозначений

Символ	Значение
$x_i, y_i, \widehat{y}_i$	Признаки, истинный ответ и прогноз для объекта i.
d, n	Число признаков и число объектов в текущем наборе.
θ	Обучаемые параметры модели.
B	Размер мини-выборки; фактический последний пакет может быть меньше.
η	Скорость обучения.
C_{in}, C_{out}	Число входных и выходных каналов свёртки.
H, W	Высота и ширина карты признаков.
TP, FP, FN	Верно найденные объекты класса, ложные срабатывания и пропуски.

В работе № 1 параметр C без индекса обозначает гиперпараметр LinearSVC; в матрице C_{ij} индексы обозначают пару истинного и предсказанного классов. Смысл определяется контекстом, а не одним символом.

Приложение Б. Шаблон отчёта

Шаблон задаёт структуру содержания. Требования к оформлению титульного листа, объёму и сроку сдачи сообщает преподаватель. Заполните поля своими сведениями; таблицы и графики включайте по смыслу исследования.

Сведения о работе

Дисциплина: _____

Лабораторная работа № ____. Название: _____

Выполнил(а): _____ Группа: _____

ИТМО ID: _____ Проверил(а): _____

Дата: _____ Версия материалов: _____

1. Цель и данные

Сформулируйте цель своими словами. Укажите источник и версию файлов, объект наблюдения, признаки и целевой ответ. Приведите размер набора и проверки его состава. Объясните, какие данные доступны при применении модели.

2. Условия эксперимента

Опишите разбиение и способ исключения утечки. Укажите среду, seed, модель, основную метрику выбора и бюджет. Для каждого опыта сформулируйте ожидаемый эффект изменения параметра до запуска. Дополнительные опыты явно отделяйте от исходного плана.

3. Результаты и разбор ошибок

Используйте таблицу следующего вида; численные ячейки заполняются после выполнения опытов.

Опыт	Что изменено	Метрика на указанной части	Наблюдение
Исходный	Конфигурация модели	Значение и единицы	Что видно по результату
Сравнение	Один фактор либо явно заданная комбинация	Та же метрика и часть	Изменение относительно исходного

Сошлитесь на диагностические графики и разберите конкретные ошибки. При неудачном запуске укажите причину, а не подставляйте нулевое значение метрики.

4. Выводы и приложение к отчёту

Ответьте, достигнута ли цель, что подтверждено результатами и чем ограничен вывод. Приложите ноутбук, конфигурации, таблицы опытов и необходимые файлы модели. Перед сдачей проверьте выполнение ноутбука сверху вниз и соответствие его результатов таблицам отчёта.